AI Sentiment Analysis

A Teaching Note for Natural Language Processing

This note accompanies the **AI Sentiment Analysis** Streamlit application (`app.py` / `sentiment_engine.py`). It explains the underlying NLP concepts, the algorithm used in the app, and includes discussion questions and exercises for classroom use.

Topic	Covered
Tokenization	Yes
Lexicon-based sentiment scoring	Yes
Negation & intensifier handling	Yes
VADER algorithm (industry-standard model)	Yes
Evaluation metrics (accuracy, precision, recall, F1)	Yes
Discussion questions & exercises	Yes

1. What Is Sentiment Analysis?

Sentiment analysis (also called opinion mining) is the task of automatically determining the emotional tone behind a piece of text — typically classifying it as **positive**, **negative**, or **neutral**. It's one of the most common introductory applications in Natural Language Processing (NLP) because it touches nearly every core NLP concept: tokenization, lexicons, context, and evaluation.

Real-world applications include analyzing product reviews, social media posts, customer support tickets, survey responses, and news coverage.

Two broad approaches

- **Lexicon-based (rule-based):** Uses a dictionary of words pre-scored with a sentiment value ("valence"). A sentence's score is computed by combining the scores of its words, adjusted by rules for negation, intensifiers, and punctuation. Fast, interpretable, no training data needed — but limited by dictionary coverage and blind to context it wasn't designed for (e.g. sarcasm).
- **Machine-learning based:** A model (logistic regression, an LSTM, or a transformer like BERT) learns to predict sentiment from labeled training examples. Can capture more context and nuance, but needs training data and is less transparent about *why* it made a prediction.

The companion app implements the lexicon-based approach twice: once as a small "from scratch" analyzer for teaching, and once via VADER, a well-known production-grade lexicon model.

2. The NLP Pipeline Used in the App

Step 1: Tokenization

Split raw text into individual words ("tokens"). Example: "I don't like it" → ["i", "don't", "like", "it"].

Step 2: Lexicon lookup

Check each token against a dictionary of words with known positive or negative scores (e.g. "great" = +2.0, "terrible" = -2.5).

Step 3: Negation handling

If a negation word ("not", "never", "don't"...) appears shortly before a sentiment word, flip that word's score. This is why "not good" scores negatively even though "good" alone is positive.

Step 4: Intensifiers & diminishers

Words like "very" or "extremely" scale a nearby sentiment score up; words like "slightly" scale it down.

Step 5: Aggregation & normalization

Sum all token scores, then compress the result into a fixed range (commonly -1 to +1) so scores are comparable across sentences of different lengths.

Why this order matters

Negation and intensifier handling must happen relative to the *position* of each sentiment word, which is only possible after tokenization has produced an ordered list of words. This is a good moment to emphasize to students that NLP pipelines are sequential: each step depends on the structure produced by the step before it.

3. Deep Dive: VADER

VADER (Valence Aware Dictionary and sEntiment Reasoner) was introduced by Hutto and Gilbert in 2014, specifically tuned for short, informal text such as tweets and reviews. It is included in NLTK and widely used in both industry and research as a fast baseline sentiment tool.

VADER's four output scores

Score	Meaning
neg	Proportion of the text classified as negative
neu	Proportion of the text classified as neutral
pos	Proportion of the text classified as positive
compound	A single normalized score in [-1, 1] summarizing overall sentiment

The compound score formula

VADER sums the valence score of every word in the text (call this sum x), applying heuristics along the way for punctuation emphasis ("!!!"), capitalization ("GREAT"), degree modifiers, and negation. That sum is then normalized to a fixed range using:

```
compound = x / sqrt(x² + α), where α = 15
```

The constant alpha (15) is a normalization parameter chosen so the compound score approaches, but never quite reaches, -1 or +1 for very strongly worded text. Classification thresholds (VADER's documented defaults) are:

- compound $\geq 0.05 \rightarrow$ Positive
- compound $\leq -0.05 \rightarrow$ Negative
- otherwise $\rightarrow$ Neutral

4. Evaluating a Sentiment Classifier

If your class has (or creates) a labeled dataset — text paired with a human judgment of positive/negative/neutral — you can measure how well an analyzer performs using standard classification metrics:

Metric	Question it answers
Accuracy	What fraction of all predictions were correct?
Precision	Of the texts predicted positive, how many really were?
Recall	Of the texts that really were positive, how many did we catch?

F1 score	A balance between precision and recall

This is a natural extension exercise: have students hand-label 30-50 rows of the sample dataset, run both analyzers, and compute these metrics for each.

5. Limitations of Lexicon-Based Sentiment Analysis

- **Sarcasm and irony:** "Oh great, another Monday" reads as positive to a lexicon but is clearly negative in context.
- **Domain-specific meaning:** "unpredictable" is negative for a car's brakes but can be positive for a movie plot.
- **Lexicon coverage:** Any word not in the dictionary contributes nothing, no matter how sentiment-laden it is (e.g. slang, misspellings, new words).
- **Negation scope:** Simple negation windows can misfire on longer or more complex sentences, e.g. "not only was it good, it was amazing."

These limitations are exactly why more advanced approaches (machine learning, transformer-based models) exist — but understanding the rule-based approach first makes those advanced methods much easier to grasp later.

6. Discussion Questions

- Why might two different sentiment tools disagree on the same sentence? Give a concrete example.
- What are the trade-offs between a small, transparent lexicon and a large, well-tuned one like VADER's?
- How would you adapt a sentiment analyzer for a specific domain (e.g. restaurant reviews vs. financial news)?
- Why can't a purely lexicon-based approach reliably detect sarcasm? What kind of information would be needed?
- If you had to choose one metric (accuracy, precision, recall, or F1) to optimize for a spam-detection-like use case, which would you pick and why?

7. Suggested Exercises

- **Beginner:** Add 10 new words to the `POSITIVE_WORDS`/`NEGATIVE_WORDS` dictionaries in `sentiment_engine.py` and observe how scores change on the Analyze Text tab.
- **Intermediate:** Find or write five sentences where the Simple Lexicon and VADER analyzers disagree. Explain why for each, using the Compare Methods tab.
- **Intermediate:** Upload a CSV of real reviews (or use `sample_reviews.csv`) on the Batch tab and discuss the resulting sentiment distribution.
- **Advanced:** Extend `sentiment_engine.py` with a third analyzer using a Hugging Face transformer sentiment pipeline, and add it to the Compare tab.

- **Advanced:** Hand-label a small dataset and compute accuracy, precision, recall, and F1 for each analyzer.

8. Glossary

Term	Definition
Token	A single unit of text after splitting, usually a word
Tokenization	The process of splitting text into tokens
Lexicon	A dictionary mapping words to pre-assigned sentiment scores
Valence	The intrinsic positive/negative charge of a word
Negation	A word (e.g. "not") that reverses the sentiment of nearby words
Compound score	A single normalized score summarizing overall sentiment
Corpus	A large collection of text used for analysis or model training

---

Companion files: app.py, sentiment_engine.py, sample_reviews.csv, README.md